LiteCDN — Content Delivery Network (CDN) Prototype

Student Details

- **Group No:** 14
- **Group Members:**
  Bhavishy Agrawal (B24CS1023) Hayagriv Desikan (B24CS1092)
  Husain Mohammad Ali (B24CS1084) Jimeet Shastri (B24CS1034)
- **Course Instructor:** Dr. Romi Banerjee

PART A: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Prepared by: *Group 14*
Organization: *Indian Institute of Technology, Jodhpur*

Revision History

Group	Date	Reason for Changes	Version
14	21/01/2026	Initial SRS creation	1.0
14	02/02/2026	Requirement Refinement and Traceability Integration	2.0
14	20/02/2026	Design Progress and	3.0

		Implementation Update	
14	02/03/2026	Design alignment after Sprint 1	4.0

TABLE OF CONTENTS

1. Introduction

1.1 Purpose
1.2 Document Conventions
1.3 Intended Audience and Reading Suggestions
1.4 Product Scope
1.5 References
1.6 Software Engineering Methodology (Essence-Inspired Approach)

2. Overall Description

2.01 Product Perspective
2.02 Product Functions

2.03 User Classes and Characteristics
2.04 Operating Environment
2.05 Design and Implementation Constraints
2.06 User Documentation
2.07 Assumptions and Dependencies
2.08 Project Roles and Responsibilities (Alpha Managers)
2.09 Project Management and Task Tracking
2.10 Design and Coding Standards

3. External Interface Requirements
3.1 User Interfaces
3.2 Hardware Interfaces
3.3 Software Interfaces
3.4 Communication Interfaces

4. System Features
4.1 Feature 1: Request Routing
4.1.1 Description and Priority
4.1.2 Stimulus/Response Sequences
4.1.3 Functional Requirements
4.2 Feature 2: Edge Caching
4.2.1 Description and Priority
4.2.2 Functional Requirements
4.3 Feature 3: Cache Management

4.3.1 Description and Priority
4.3.2 Functional Requirements

5. Other Nonfunctional Requirements
5.1 Performance Requirements
5.2 Safety Requirements
5.3 Security Requirements
5.4 Software Quality Attributes
5.5 Business Rules

6. Other Requirements
7. Appendix
7.1 Appendix A
7.2 Appendix B
7.3 Appendix C

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document specifies the requirements for the LiteCDN experimental Content Delivery Network prototype, featuring weighted routing and segmented edge caching mechanisms.

The document defines functional and non-functional requirements and serves as a reference for system design, UML modeling, implementation, and validation.

1.2 Document Conventions

- **SHALL** indicates a mandatory requirement
- **SHOULD** indicates a recommended requirement
- **MAY** indicates an optional requirement
- Requirement IDs follow the format **REQ-X**

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Course instructors and evaluators
- Student developers
- Project reviewers

Readers should begin with **Section 1 & 2** for system overview, followed by **Section 4 (System Features)** for detailed requirements.

1.4 Product Scope

The LiteCDN Prototype is an educational and experimental software system designed to demonstrate structured content delivery using centralized routing intelligence and segmented edge caching.

The system incorporates latency-load weighted routing with controlled exploration and a three-tier segmented cache (New/Unproven, Popular/Proven, MissAware) to study routing-caching tradeoffs under controlled stress conditions.

The system aims to:

- Reduce latency through intelligent edge selection
- Balance load across edge servers using weighted scoring
- Maintain structural cache stability through segment-based eviction
- Demonstrate cache hit/miss behavior and origin-level miss-aware promotion

The prototype prioritizes architectural clarity and reproducibility over production-scale optimization.

1.5 References

- IEEE Software Requirements Specification Standard (ISO/IEC/IEEE 29148:2018)
- Course Handouts: Requirements Engineering & UML Diagrams
- UML models created for this project

1.6 Software Engineering Methodology (Essence-Inspired Approach)

[Essence content]

Justification for Essence-Inspired Approach:The SEMAT Essence framework was applied conceptually to provide a structured perspective for assessing project progress and engineering completeness. Essence Alphas (Stakeholders, Requirements, Software System, Work, Team, and Way of Working) guided evaluation of requirement maturity, architectural evolution through UML artifacts, work progression, and team coordination.

Essence was used as a lightweight evaluative approach rather than a rigid methodology, improving consistency between requirements, design models, and development activities.

2. Overall Description

2.01 Product Perspective

The LiteCDN Prototype is a centralized-routing distributed system consisting of:

- Client
- CDNSystem (request orchestrator)
- RoutingService (latency-load weighted routing with exploration)
- Six EdgeServer instances with segmented in-memory caches
- OriginServer (authoritative content + miss tracking)
- AdminAPI and monitoring interface

2.02 Product Functions

The system provides the following major functions:

- Accept content requests from users
- Route requests to appropriate edge servers
- Serve cached content when available
- Fetch content from origin server on cache miss
- Cache content for future requests
- Weighted routing using latency-load scoring
- Controlled exploration to prevent edge dominance
- Segmented cache with structural eviction policy
- Centralized origin-level miss tracking to support MissAware promotion.

2.03 User Classes and Characteristics

User Class	Characteristics
End User	Requests content, no technical expertise
Administrator	Monitors system behavior

2.04 Operating Environment

- OS: Windows / Linux / macOS
- Network: LAN / WAN
- Protocol: HTTP
- Language: Node.js/ FastAPI/Express.js (prototype level)

2.05 Design and Implementation Constraints

- No real DNS-based routing (plans on hosting in future)
- Limited number of edge servers
- Prototype-scale dataset
- Open-source tools only
- Centralized routing architecture (no distributed consensus or inter-edge synchronization).

2.06 User Documentation

- Project report
- UML diagrams
- README file

2.07 Assumptions and Dependencies

- Network connectivity is available
- Content is static
- Edge servers are reachable

2.08 Project Roles and Responsibilities (Alpha Managers)

Responsibility Owner – Architecture, UML, and Implementation-**Jimeet Shastri**

- Responsible for overall system architecture

- Maintains consistency across UML diagrams
- Oversees implementation of core modules

Responsibility Owner – Requirements, Documentation, and Integration-**Bhavishy Agrawal**

- Owns SRS versions and requirement traceability
- Maintains project documentation
- Oversees system integration and validation against requirements

---

2.09 Project Management and Task Tracking

Trello was adopted as the primary task tracking and coordination tool to support structured project management throughout the LiteCDN prototype development. A Kanban-style board (To Do, In Progress, Review, Done) was maintained to reflect sprint-wise implementation progress and requirement coverage.

Each functional requirement (REQ-1 to REQ-15) was mapped to one or more Trello cards to ensure traceability between the SRS, UML artifacts, and implementation tasks. Sprint 1 focused on establishing end-to-end connectivity and validating architectural integration. Advanced routing intelligence, segmented caching mechanisms, and cache management policies are scheduled for subsequent sprints in alignment with architectural dependency planning.

Trello's card-based workflow enabled systematic decomposition of features (e.g., Routing, Edge Caching, Cache Management) into manageable implementation units. This structure ensured clear responsibility allocation, transparent progress tracking, and controlled sprint execution. The Review column was used for integration validation and functional verification before tasks were moved to Done.

Trello Kanban Board Status After Sprint 1 (Requirement-Mapped View):

TO DO	IN PROGRESS	REVIEW	DONE
REQ-4: Implement ε-Greedy Exploration	REQ-2: Implement Latency–Load Scoring	—	REQ-1: Accept Client Requests
REQ-5: Implement Dynamic Load Reflection	REQ-3: Implement Score-Based Edge Selection (Full Version)	—	REQ-6: Check Cache Before Origin Call (Basic)
REQ-9: Insert into New/Unproven Segment	REQ-11: Implement TTL Assignment	—	REQ-7: Serve Cached Content on Hit
REQ-10: Implement Segment Promotion Logic	REQ-12: Implement Lazy Expiration	—	REQ-8: Fetch from Origin on Miss
REQ-13: Implement LRU Eviction per Segment		—	End-to-End Architectural Validation
REQ-14: Implement Inter-Segment Eviction Policy		—	Logging and Basic Error Handling
REQ-15: Implement MissAware Promotion Threshold		—	

Note: Detailed implementation-level task decomposition and card descriptions are maintained within the Trello board to support granular sprint tracking and integration monitoring. At the time of documentation, no tasks were pending review, as all Sprint 1 items had completed validation and were transitioned to Done.

2.10 Design and Coding Standards

Language Assumptions

The CDN prototype is implemented using Node.js (backend) and React.js (administrative interface). The following conventions apply independent of the chosen language.

Naming Conventions

- Class and module names use *PascalCase*.
- Function names use *camelCase* and describe system actions.
- Variable names use *camelCase* and represent stored data.
- Constants use *UPPER_SNAKE_CASE*.

Commenting Standards

- Each source file includes a brief header comment describing its purpose.
- Non-trivial functions include comments describing their behavior, inputs, and outputs.

- Inline comments are used only when necessary to clarify complex logic.

Project Structure

- Source code is organized into directories based on system responsibility (e.g., routing, edge handling, origin services, shared utilities).

Function vs Variable Naming

Function names represent operations or behavior, while variable names represent data or state. This distinction improves readability and helps clearly separate system logic from stored information.

---

3. External Interface Requirements

3.1 User Interfaces

- Web browser (Admin interface for monitoring, routing parameter adjustment, and health inspection)
- Command-line HTTP client (for testing content request flow and API endpoints)

3.2 Hardware Interfaces

- Standard computing devices (PC/Laptop)

3.3 Software Interfaces

- Node.js runtime with Express framework (backend services)
- In-memory segmented cache module within EdgeServer
- RoutingService module implementing latency-load scoring
- Logging module for request and cache operations

3.4 Communications Interfaces

- HTTP over TCP/IP for client-to-CDN and inter-component communication
- REST-style endpoints for administrative control and health monitoring

---

4. System Features

4.1 Feature 1: Request Routing

4.1.1 Description and Priority

Routes incoming client requests to an appropriate edge server.
Priority: High

4.1.2 Stimulus/Response Sequences

- Client sends request.
- RoutingService computes a score for each available edge server based on latency and load parameters.
- RoutingService selects an edge server using weighted scoring with controlled exploration.
- Request is forwarded to the selected edge server.

4.1.3 Functional Requirements

- **REQ-1:** The system SHALL accept client requests.
- **REQ-2:** The system SHALL compute a routing score for each available edge server using a weighted combination of latency and load parameters.
- **REQ-3:** The system SHALL select an edge server based on the computed routing scores.
- **REQ-4:** The routing mechanism SHALL support bounded exploration to prevent persistent selection of a single edge server under burst traffic.
- **REQ-5:** The routing mechanism SHALL dynamically reflect edge load variations during server selection.

4.2 Feature 2: Edge Caching

4.2.1 Description and Priority

Implements a segmented in-memory cache at each EdgeServer to reduce latency and control object lifecycle across New/Unproven, Popular/Proven, and MissAware segments.
Priority: High

4.2.2 Functional Requirements

- REQ-6: The system SHALL check the appropriate cache segment before contacting the OriginServer.
- REQ-7: The system SHALL serve cached content on cache hit.
- REQ-8: Upon a cache miss at the selected edge server, the system SHALL fetch content from the OriginServer.
- REQ-9: The system SHALL insert fetched content into the New/Unproven segment by default.
- REQ-10: The system SHALL support promotion of objects between segments based on access patterns and origin-level miss tracking.

4.3 Feature 3: Cache Management

4.3.1 Description and Priority

Manages cache expiration using TTL.
Priority: Medium

4.3.2 Functional Requirements

- **REQ-11**:

  The system SHALL associate a configurable Time To Live (TTL) with each cached object at insertion.

  -Segment-specific default TTL values:

  -New/Unproven segment: 60 seconds

  -Popular segment: 180 seconds

  -MissAware segment: 300 seconds

  -Maximum allowable TTL: 600 seconds.

  -TTL values MAY be adjusted for experimental evaluation.

- **REQ-12**:

  The system SHALL invalidate expired cache entries.

- **REQ-13**:

  Eviction SHALL occur within each cache segment using a Least Recently Used (LRU) policy.

- **REQ-14**:

When inter-segment eviction is required, eviction priority SHALL follow:

New/Unproven → MissAware → Popular/Proven.

- **REQ-15**:

  Placement into the MissAware segment SHALL be triggered based on repeated origin-level cache misses exceeding a defined threshold.

  Expiration Policy

  -Expiry time SHALL be computed as:

  -Expiry time = insertion time + TTL

  -Expired entries SHALL be treated as cache misses.

  -Expiration SHALL be enforced via lazy validation upon access.

  -The system MAY perform periodic cleanup of expired entries.

---

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Cached responses SHOULD have lower latency than origin responses.
- The system SHOULD support up to 50 concurrent HTTP requests under prototype cloud deployment conditions
- The routing mechanism SHOULD distribute load across edge servers to reduce persistent load imbalance under burst traffic.
- The segmented cache design MAY trade marginal hit-rate reduction for improved structural eviction stability.

5.2 Safety Requirements

- The system SHALL prevent unintended modification or corruption of cached content during routing and cache operations.

5.3 Security Requirements

- The system SHOULD validate incoming HTTP requests to prevent malformed input.
- Administrative endpoints SHALL restrict configuration and monitoring access to authorized users.

5.4 Software Quality Attributes

- Reliability:The system SHOULD maintain consistent routing and caching behavior under sustained request load.

- Scalability:The architecture SHOULD support extension to additional edge servers without redesigning core routing logic.

- Maintainability:The modular separation of RoutingService, EdgeServer, and OriginServer SHALL support independent evolution of routing and caching policies.

- Portability:The system SHOULD be deployable on standard operating systems using open-source tools without specialized hardware.

5.5 Business Rules

- Only administrators can monitor or manage the system.
- End users can only request content.

---

6. Other Requirements

- Logging of routing decisions, cache hits/misses, and segment transitions
- Logging of routing parameter adjustments (α, β, ε)
- Error handling for unavailable edge or origin servers
- Health monitoring for edge load and availability

---

7. Appendix

Appendix A: Glossary

- α-β Routing – Weighted latency-load scoring mechanism for edge selection
- ε-Greedy Exploration – Controlled probabilistic selection of non-optimal edge to prevent stagnation
- MissAware Segment – Cache segment reserved for objects promoted based on repeated origin-level misses
- LRU – Least Recently Used eviction policy

Appendix B: Analysis Models

- Use Case Diagram
- Class Diagram

- Component Diagram
- Composite Structure Diagram
- Sequence Diagram
- Activity Diagram
- Timing Diagram
- Deployment Diagram

Appendix C: To Be Determined List

- TBD-1: Dynamic or adaptive TTL based on access frequency
- TBD-2: Distributed cache coherence for multi-region scalability
- TBD-3: Automated routing parameter tuning under varying traffic patterns

---

PART B: UML DIAGRAMS

TABLE OF CONTENTS

1. Use Case Diagram
2. Class Diagram

3. Component Diagram
4. Composite Structure Diagram
5. Sequence Diagram
6. Activity Diagram
7. Timing Diagram
8. Deployment Diagram

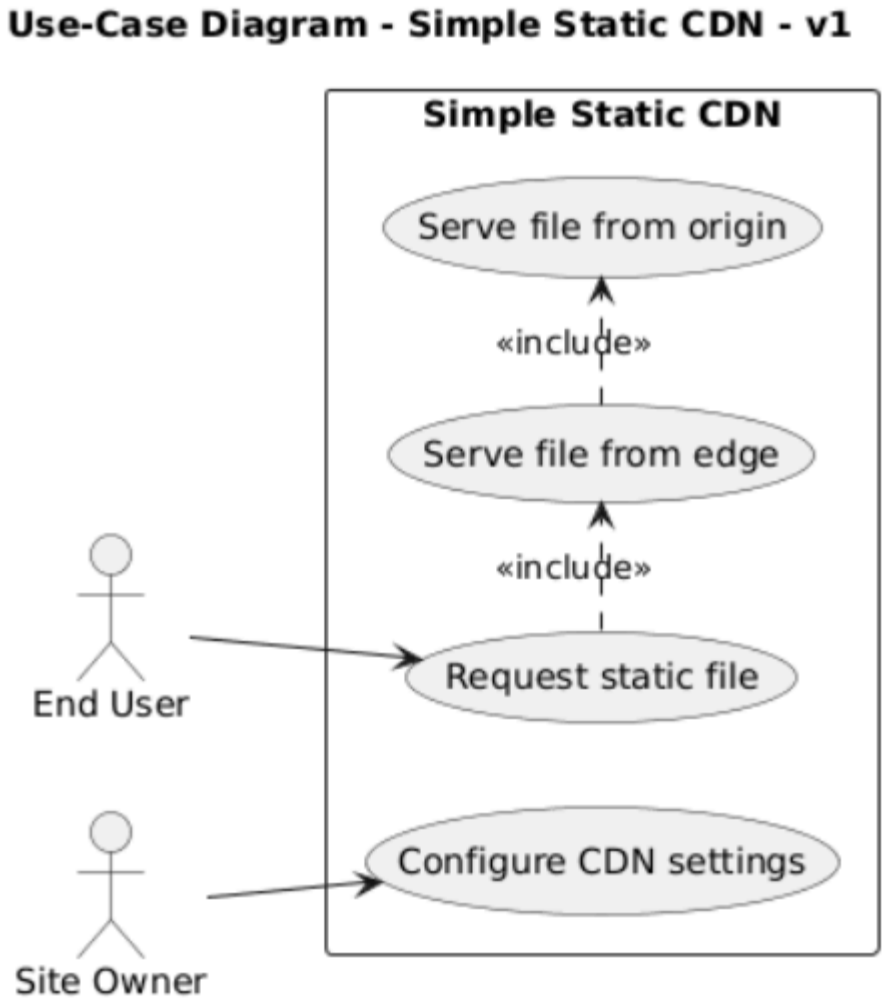
1. Use Case Diagram

Purpose: Shows interactions between users and the CDN system.

Actors:

- End User
- Administrator

Diagram:



---

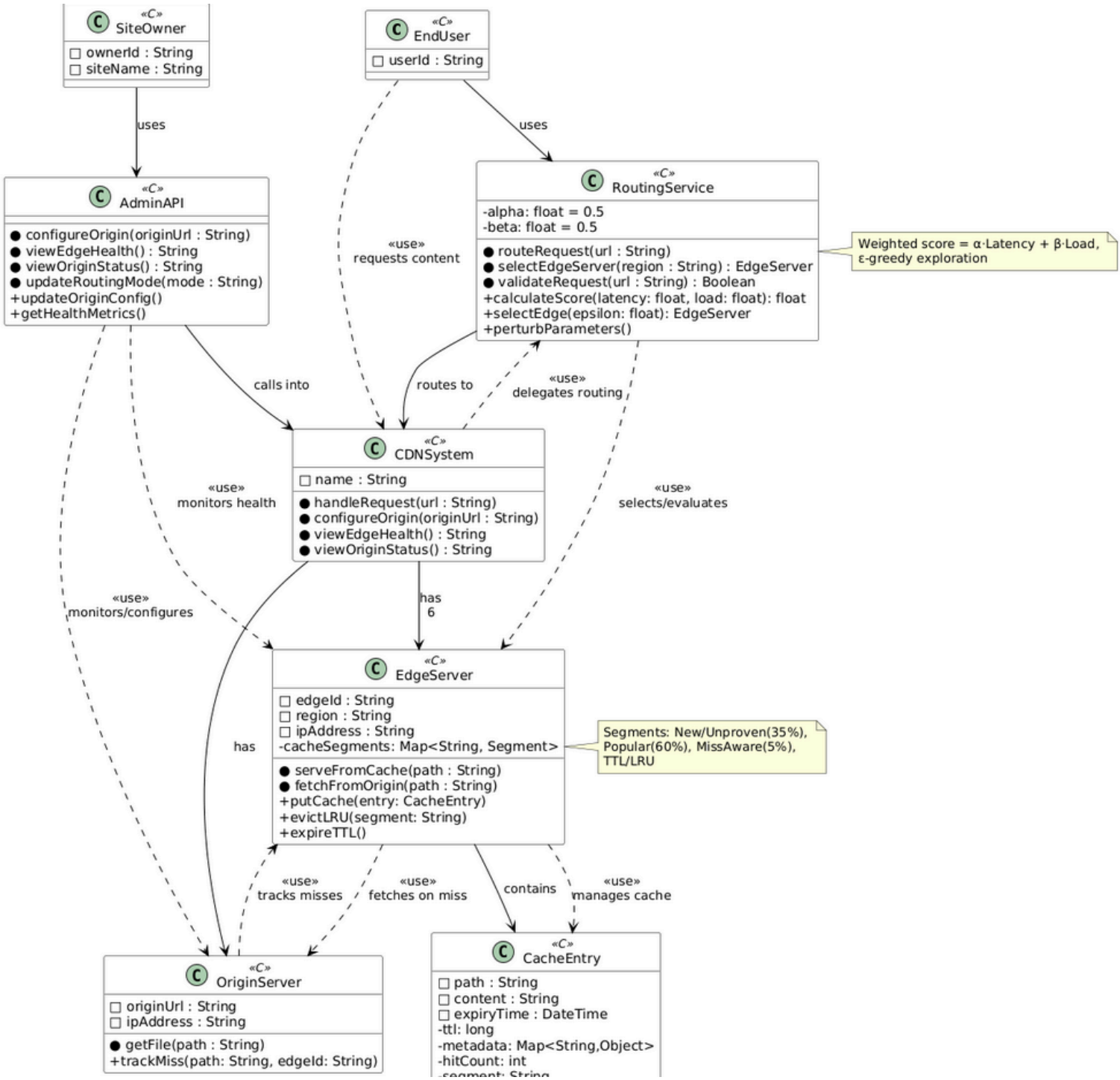
2. Class Diagram

Purpose: Represents static structure of the system, including classes and relationships.

Key Classes:

- Client
- Router
- EdgeServer
- Cache
- OriginServer

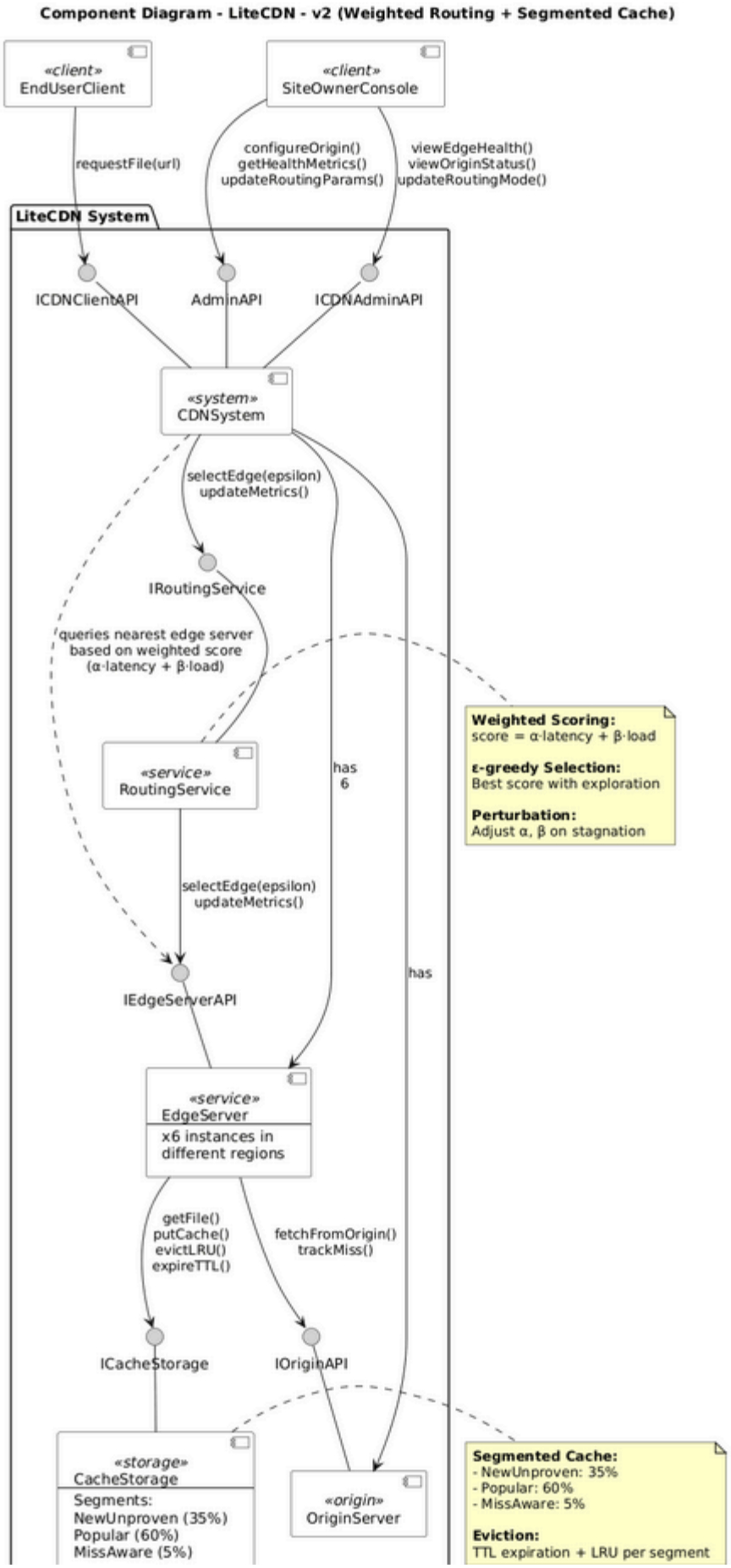
Diagram:


«C» SiteOwner
ownerId : String
siteName : String
uses
«C» EndUser
userId : String
uses
«C» AdminAPI
configureOrigin(originUrl : String)
viewEdgeHealth() : String
viewOriginStatus() : String
updateRoutingMode(mode : String)
+updateOriginConfig()
+getHealthMetrics()
«use» requests content
«C» RoutingService
-alpha: float = 0.5
-beta: float = 0.5
routeRequest(url : String)
selectEdgeServer(region : String) : EdgeServer
validateRequest(url : String) : Boolean
+calculateScore(latency: float, load: float): float
+selectEdge(epsilon: float): EdgeServer
+perturbParameters()
Weighted score = α·Latency + β·Load, ε-greedy exploration
calls into
routes to
«use» delegates routing
«C» CDNSystem
name : String
handleRequest(url : String)
configureOrigin(originUrl : String)
viewEdgeHealth() : String
viewOriginStatus() : String
«use» monitors health
«use» selects/evaluates
«use» monitors/configures
has 6
«C» EdgeServer
edgeId : String
region : String
ipAddress : String
-cacheSegments: Map<String, Segment>
serveFromCache(path : String)
fetchFromOrigin(path : String)
+putCache(entry: CacheEntry)
+evictLRU(segment: String)
+expireTTL()
Segments: New/Unproven(35%), Popular(60%), MissAware(5%), TTL/LRU
has
«use» tracks misses
«use» fetches on miss
contains
«use» manages cache
«C» OriginServer
originUrl : String
ipAddress : String
getFile(path : String)
+trackMiss(path: String, edgeId: String)
«C» CacheEntry
path : String
content : String
expiryTime : DateTime
-ttl: long
-metadata: Map<String,Object>
-hitCount: int

3. Component Diagram

Purpose: Illustrates software components and their dependencies.

Diagram:


Component Diagram - LiteCDN - v2 (Weighted Routing + Segmented Cache)
«client»
EndUserClient
«client»
SiteOwnerConsole
requestFile(url)
configureOrigin()
getHealthMetrics()
updateRoutingParams()
viewEdgeHealth()
viewOriginStatus()
updateRoutingMode()
LiteCDN System
ICDNClientAPI
AdminAPI
ICDNAdminAPI
«system»
CDNSystem
selectEdge(epsilon)
updateMetrics()
IRoutingService
queries nearest edge server
based on weighted score
(α·latency + β·load)
Weighted Scoring:
score = α·latency + β·load
ε-greedy Selection:
Best score with exploration
Perturbation:
Adjust α, β on stagnation
«service»
RoutingService
has
6
selectEdge(epsilon)
updateMetrics()
has
IEdgeServerAPI
«service»
EdgeServer
x6 instances in
different regions
getFile()
putCache()
evictLRU()
expireTTL()
fetchFromOrigin()
trackMiss()
ICacheStorage
IOriginAPI
«storage»
CacheStorage
Segments:
NewUnproven (35%)
Popular (60%)
MissAware (5%)
«origin»
OriginServer
Segmented Cache:
- NewUnproven: 35%
- Popular: 60%
- MissAware: 5%
Eviction:
TTL expiration + LRU per segment

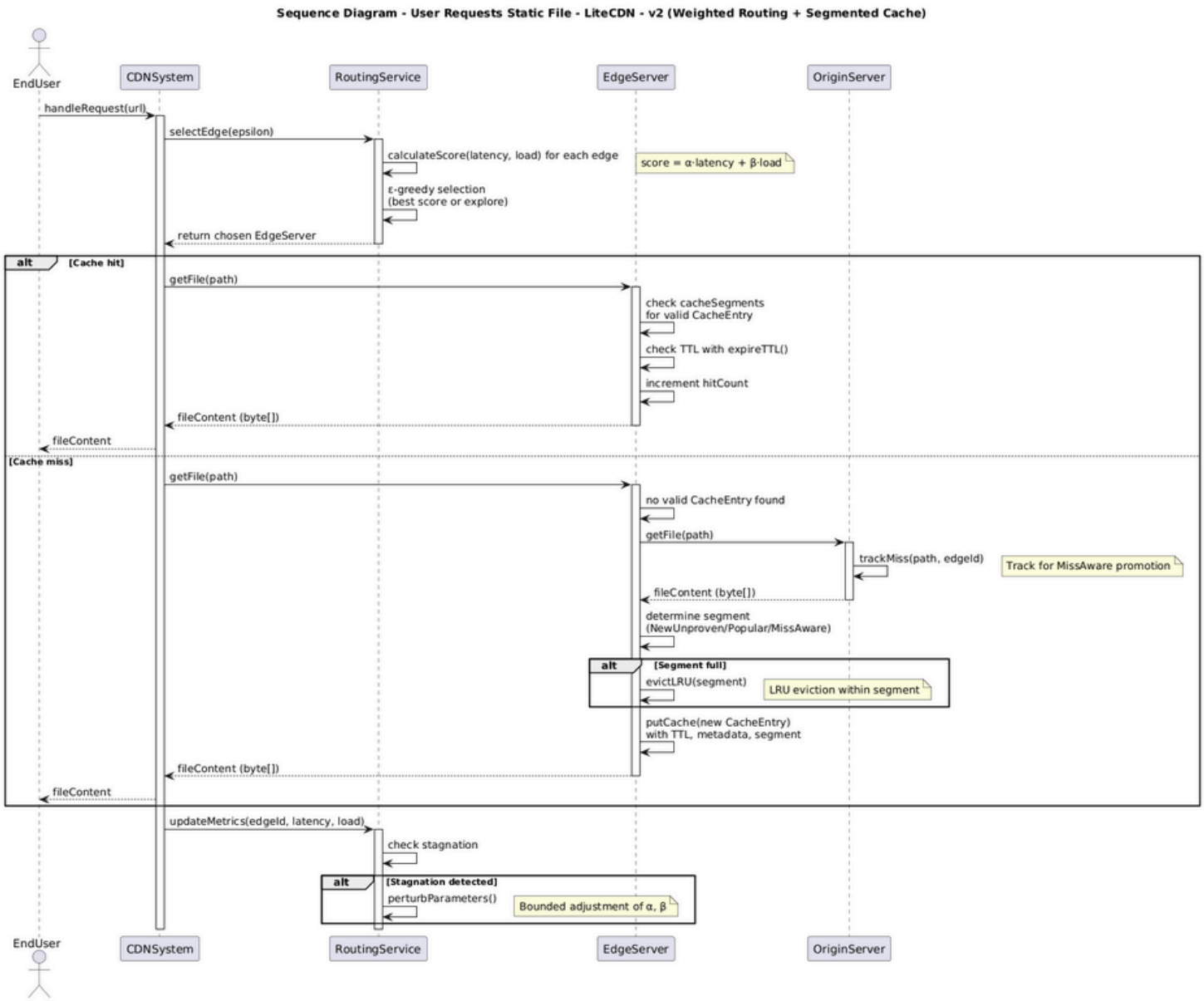
---

4. Sequence Diagram

Purpose: Demonstrates interaction over time for:

- Cache Hit scenario
- Cache Miss scenario

Diagram:

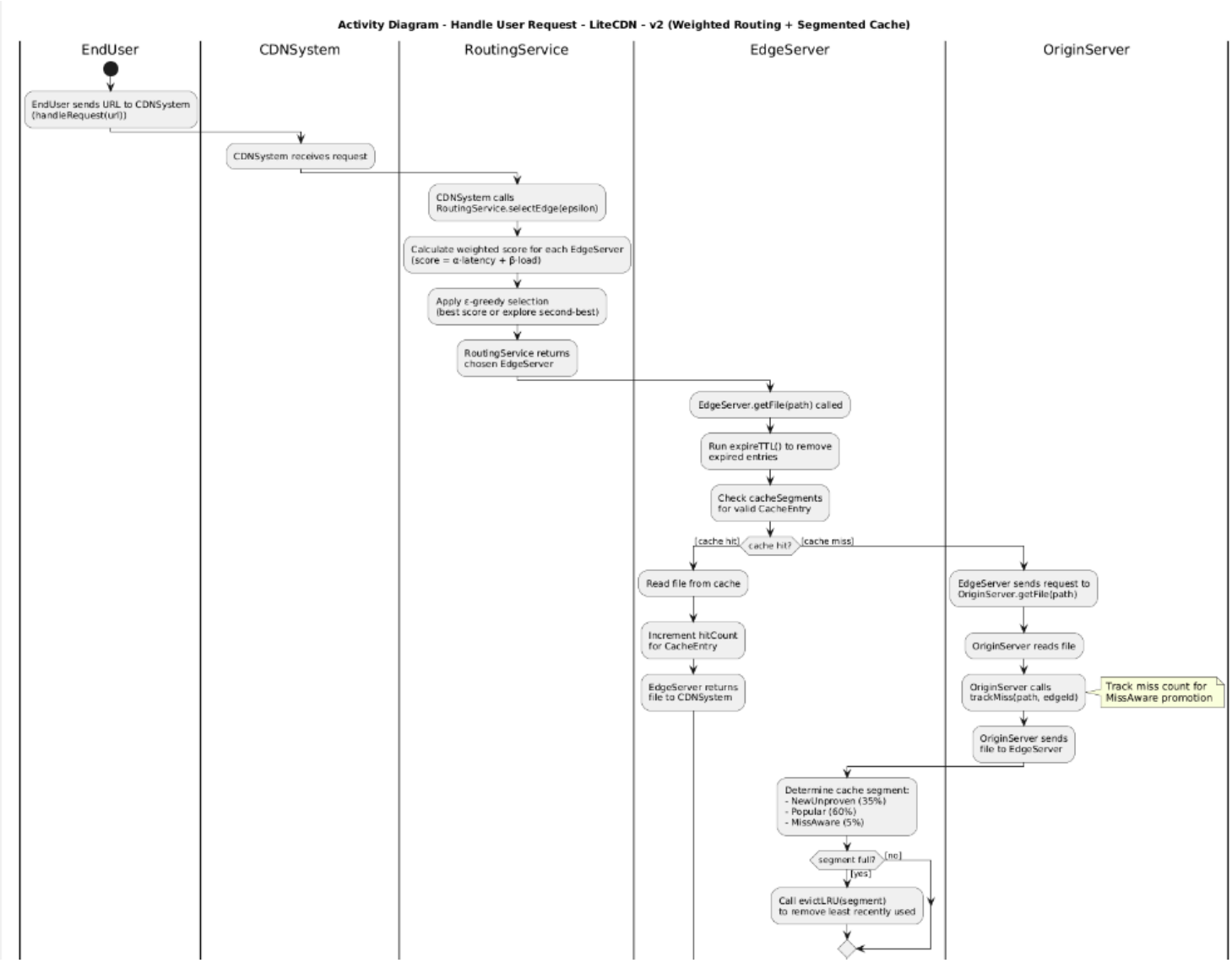


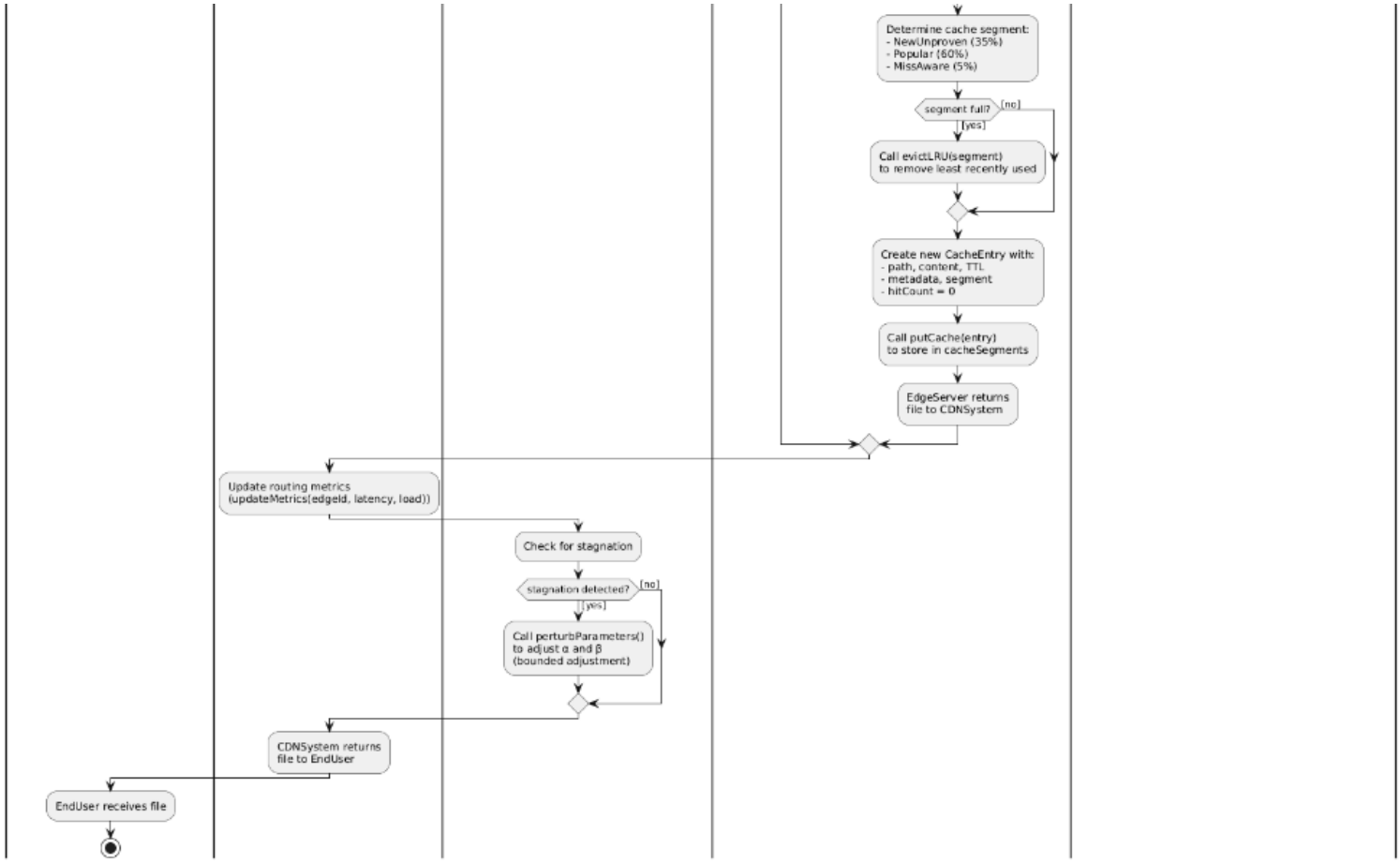
---

5. Activity Diagram

Purpose: Represents workflow of request handling and cache decision logic.

Diagram:

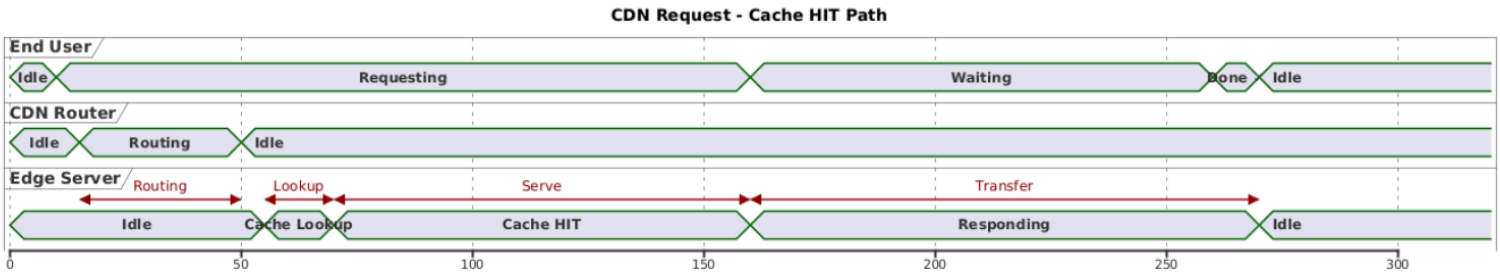

Activity Diagram - Handle User Request - LiteCDN - v2 (Weighted Routing + Segmented Cache)
EndUser
CDNSystem
RoutingService
EdgeServer
OriginServer
EndUser sends URL to CDNSystem (handleRequest(url))
CDNSystem receives request
CDNSystem calls RoutingService.selectEdge(epsilon)
Calculate weighted score for each EdgeServer (score = α·latency + β·load)
Apply ε-greedy selection (best score or explore second-best)
RoutingService returns chosen EdgeServer
EdgeServer.getFile(path) called
Run expireTTL() to remove expired entries
Check cacheSegments for valid CacheEntry
cache hit?
[cache hit]
[cache miss]
Read file from cache
Increment hitCount for CacheEntry
EdgeServer returns file to CDNSystem
EdgeServer sends request to OriginServer.getFile(path)
OriginServer reads file
OriginServer calls trackMiss(path, edgeId)
Track miss count for MissAware promotion
OriginServer sends file to EdgeServer
Determine cache segment: - NewUnproven (35%) - Popular (60%) - MissAware (5%)
segment full?
[no]
[yes]
Call evictLRU(segment) to remove least recently used



6. Timing Diagram

Purpose: Illustrates timing constraints and latency differences between cache hit and cache miss.

Diagram:

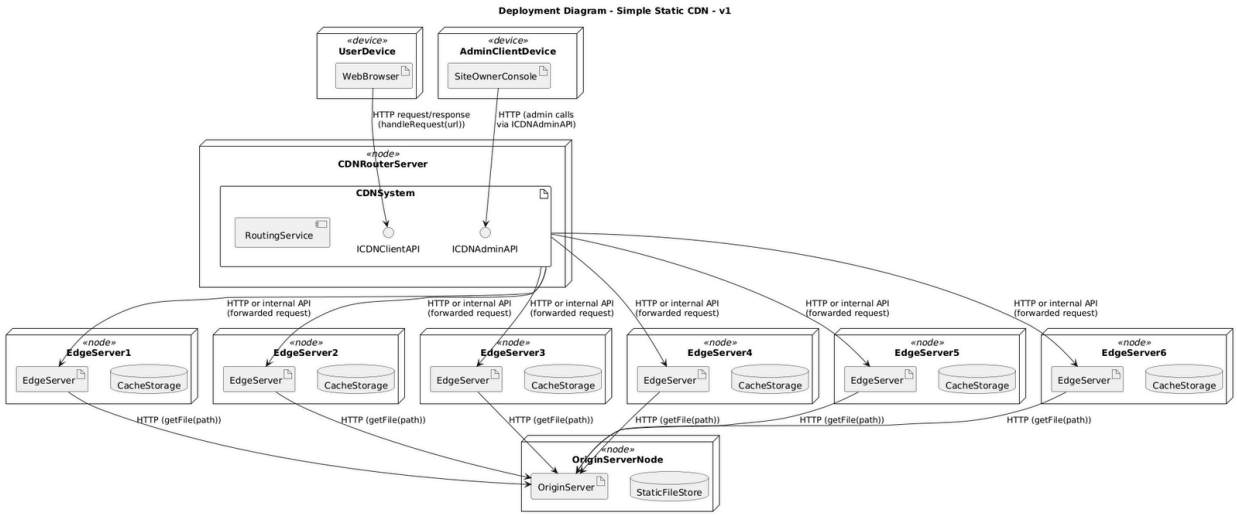


---

7. Deployment Diagram

Purpose: Shows physical deployment of system components across hardware nodes.

Diagram:



PART C: TRACEABILITY MATRIX

For readability and layout optimization, the Requirements Traceability Matrix is presented in two logically separated tables. The Requirement ID uniquely identifies and links corresponding entries across both tables. This separation does not alter the integrity or completeness of traceability.

Req ID	Requirement Name	Requirement Type	Change Type
FR-UC-01	Content Request Handling	Functional	Refined
FR-CD-01	Request Routing and Origin Fetch	Functional	Refined
FR-COMP-01	CDN Component Decomposition	Functional	Refined
FR-SEQ-01	Cache-Aware Request Flow	Functional	Refined
FR-ACT-01	Content Delivery Activity Flow	Functional	Refined
FR-DEP-01	CDN Deployment Architecture	Functional / Architectural	Refined
PR-01	Project Roles and Responsibilities	Process / Governance	New
NFR-DS-01	Design and Coding Standards	Non-Functional	New

Req ID	Requirement Description	UML / Reference Artifact	Verification Method
FR-UC-01	System shall allow an end user to request content which may be served from edge or fetched from origin	Use Case Diagram (Lab 2, Lab 3)	Use-case scenario walkthrough
FR-CD-01	System shall route client content requests to appropriate edge servers and fetch from origin if required	Class Diagram (Lab 2, Lab 3)	Class responsibility consistency review
FR-COMP-01	System shall decompose CDN functionality into routing, edge delivery, caching, and origin access components	Component Diagram (Lab 2, Lab 3)	Component interaction review
FR-SEQ-01	System shall process file requests by routing to an edge server and serving content from cache or origin	Sequence Diagram (Lab 2, Lab 3)	Sequence flow walkthrough
FR-ACT-01	System shall handle user requests by checking cache,fetching from origin on a miss, caching results, and serving content	Activity Diagram (Lab 2, Lab 3)	Activity flow consistency review
FR-DEP-01	System shall be deployed using a CDN architecture with multiple edge servers retrieving content from an origin server	Deployment Diagram (Lab 2, Lab 3)	Deployment topology review
PR-01	Project shall define roles and responsibilities to ensure architectural consistency, requirement ownership, and validation	SRS Section 2.8	Documentation review
NFR-DS-01	System implementation shall follow defined naming, commenting, and structural coding standards to ensure maintainability	SRS Section 2.9	Standards compliance review

PART D: PROJECT MANAGEMENT AND SPRINT PLAN

TABLE OF CONTENTS

1. Project Management Tools
2. Sprint Strategy and Phasing
3. Detailed Sprint Schedule
4. Essence Alpha Progression after Sprint 1
5.

1. Project Management Tools

To ensure efficient collaboration and tracking of the iterative development process, we've utilized the following tools:

- **Task Tracking (Trello/Jira):** We have adopted Trello using a Kanban-style board (To Do, In Progress, Review, Done).
  - *Logic:* Trello allows for visual management of tasks. Each "Feature" (e.g., Edge Caching) is broken down into individual cards (e.g., "Implement LRU Algorithm", "Handle Cache Miss"). This ensures no requirement is overlooked and allows Alpha Managers to track progress against the timeline.
- **Version Control (GitHub/GitLab):** Source code is managed via GitHub. Feature branches are used for developing specific modules (e.g., feature/request-router) and merged into the main branch only after passing basic integration tests.

2. Sprint Strategy and Phasing

The implementation of the LiteCDN Prototype is divided into four 2-week sprints. The sprint order follows architectural dependency: foundational connectivity first, deterministic routing and segmented caching next, controlled routing optimization thereafter, and finally system integration and stabilization.

Logic for Sprint Breakdown:

Connectivity First:

Caching and routing policies depend on stable communication between Client, CDNSystem, EdgeServer, and OriginServer. Therefore, Sprint 1 establishes end-to-end request flow and validates component interfaces before introducing routing intelligence or cache segmentation.

Core Routing and Segmented Caching:

Sprint 2 introduces the system's primary functionality: **α-β** weighted routing based on latency-load scoring and a segmented cache architecture (New 35% / Popular 60% / MissAware 5%). TTL enforcement and LRU eviction per segment are implemented at this stage to define deterministic, stable core behavior.

Controlled Routing Optimization and Observability:

Sprint 3 extends routing with **ε**-greedy exploration and bounded parameter perturbation to prevent edge stagnation under burst traffic. Centralized miss tracking at the OriginServer and administrative observability are added once the deterministic core is stable.

Integration and Stabilization:

Sprint 4 focuses on integration testing, routing-caching interaction validation, concurrency checks, UML finalization, and documentation alignment. This ensures architectural consistency and readiness for final submission.

3. Detailed Sprint Schedule

Sprint	From	To	Duration	Sprint Goal	Key Technical Tasks
Sprint 1	17 Feb 2026	2 March 2026	2 Weeks	Establish end-to-end request flow and connectivity	- Implement minimal OriginServer (serve static files) - Implement EdgeServer pass-through - Implement CDNSystem.handleRequest() - Implement RoutingService (Round Robin → Weighted) - Basic console logging
Sprint 2	3 March 2026	16 March 2026	2 Weeks	Implement segmented caching and α–β weighted routing	- Define CacheEntry structure - Implement cache hit/miss logic - Implement segment-aware TTL (New/Popular/MissAware) - Implement LRU-based eviction per segment - Cache flow validation tests -Implement latency–load scoring (Score = α·Latency + β·Load) -Add latencyMap and loadMap tracking
Sprint 3	17 March 2026	30 March 2026	2 Weeks	Add routing optimization, administrative control, and observability	- Implement AdminAPI (configureOrigin, viewEdgeHealth) - Implement health endpoints (UP/DOWN status) - Improve error handling (404/timeouts) - Validate Render deployment - Concurrency validation (~50 requests) -Implement ε-greedy edge selection -Implement bounded parameter perturbation -Add stagnationCounter logic -Implement centralized miss tracking at OriginServer
Sprint 4	31 March 2026	13 April 2026	2 Weeks	Stabilize system and finalize documentation	- Integration testing - Finalize UML diagrams (Class, Sequence, Component, Deployment) - Update SRS and traceability matrix - Performance verification - Final submission preparation

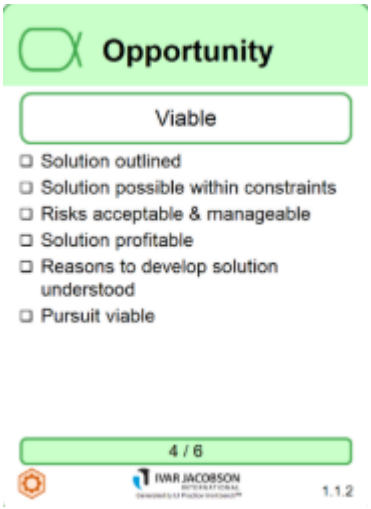
Sprint	Deliverables	Role Focus & Work Assigned	Status
Sprint 1	Working request path: Client → CDN → Edge → Origin	PO – Hayagriv: Define connectivity backlog, validate end-to-end flow SM – Husain: Sprint planning & coordination Jimeet: Design server skeleton & routing structure Bhavishy: Validate SRS against implementation	Completed
Sprint 2	Functional segmented cache (35/60/5) with α–β weighted routing	PO – Hayagriv: Prioritize caching features SM – Husain: Coordinate routing-cache integration Jimeet: Implement segmentation & update class diagram Bhavishy: Update SRS & sequence diagram for cache flow	Ongoing
Sprint 3	Routing optimization (ε-greedy + perturbation) with administrative observability	PO – Jimeet: Validate configuration & health features SM – Husain: Facilitate AdminAPI integration Hayagriv: Implement Admin endpoints Bhavishy: Update component & deployment diagrams	Planned
Sprint 4	Tested system + finalized documentation	PO – Jimeet: Validate feature completeness SM – Husain: Ensure integration stability Hayagriv: Integration tests & error handling Bhavishy: Finalize UML, traceability matrix, complete SRS	Planned

4. Essence Alpha Progression After Sprint 1

4.1 **Opportunity — Viable**

Justification:
The LiteCDN prototype problem space is clearly defined and validated. The system scope, architectural direction, and educational objectives are established. A working end-to-end request flow demonstrates technical feasibility of the proposed solution.

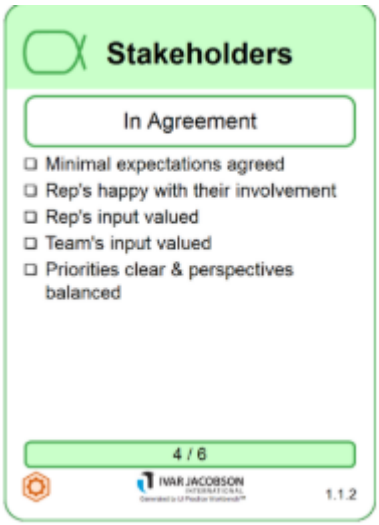
Not Yet Addressed:
Core value-driving mechanisms such as weighted routing (α–β scoring) and segmented caching have not yet been implemented. Therefore, the opportunity has not yet been fully realized in terms of delivering its intended performance and architectural benefits.



4.2 **Stakeholders — In Agreement**

Justification:
Stakeholders (course instructor and project team) share a common understanding of system objectives, sprint breakdown, and architectural direction. Roles and responsibilities are clearly defined, and Sprint 1 deliverables align with documented requirements.

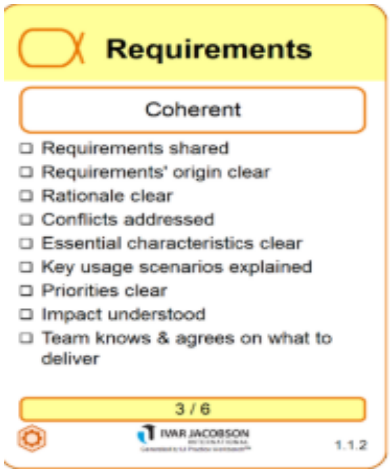
Not Yet Satisfied for Deployment:
The system is still in early implementation stages and does not yet provide complete functionality required for full stakeholder satisfaction.



4.3 **Requirements — Coherent**

Justification:
The SRS document is structured, version-controlled, and aligned with UML artifacts. Requirements are uniquely identified and traceable. Foundational requirements related to request acceptance and origin communication are partially implemented and validated.

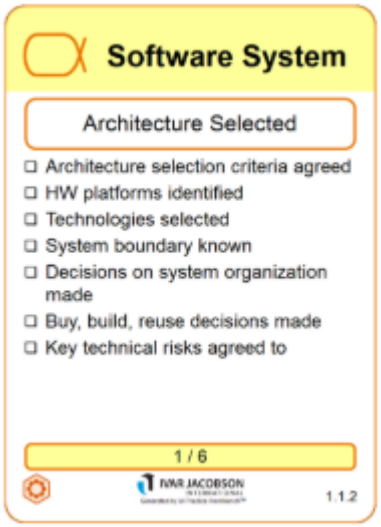
Not Yet Acceptable:
High-priority functional requirements such as weighted routing, segmented caching, TTL enforcement, and exploration mechanisms are not yet implemented or validated.



4.4 **Software System — Architecture Selected**

Justification:
The system architecture is defined and implemented at a foundational level. Core components (RoutingService, EdgeServer, OriginServer, CDNSystem) are integrated, and end-to-end communication has been validated. Deployment structure and interfaces are established.

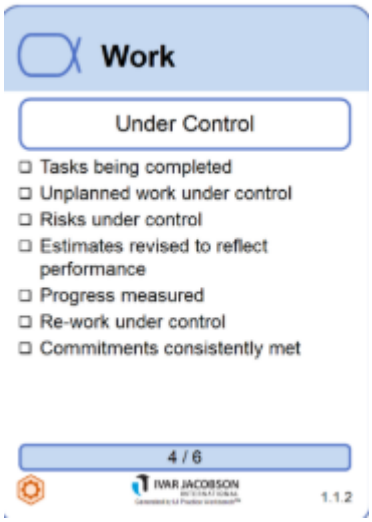
Not Yet Demonstrable:
Advanced functional behavior including latency–load scoring, ε-greedy exploration, segmented caching, and eviction policies are not yet operational.



4.5 **Work — Under Control**

Justification:
Sprint planning is structured and tracked through Trello. Version control practices are established using Git with feature branches. Sprint 1 objectives were completed as planned, and integration baseline is stable.

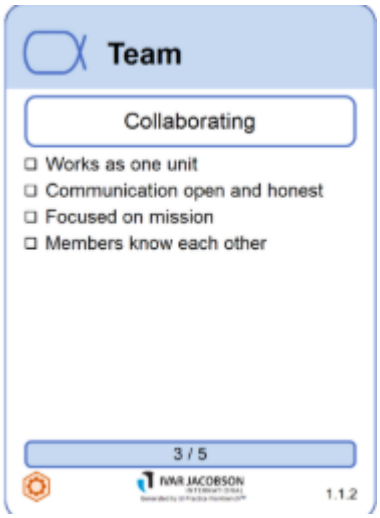
Not Yet Concluded:
Core feature implementation remains pending in subsequent sprints.



4.6 **Team — Collaborating**

Justification:
Team members have clearly defined responsibilities (Architecture/UML and Requirements/Documentation leadership). Coordination between design and implementation activities is active, and documentation reflects collaborative effort.

Not Yet Performing:
The team has not yet reached peak productivity under complex feature implementation, as core routing and caching mechanisms begin in Sprint 2.



4.7 **Way of Working — In Place**

Justification:
Development practices including sprint-based iteration, Kanban tracking, Git workflow, coding standards, and Essence-based evaluation are defined and actively followed. The team operates under a structured and repeatable process.

Not Yet Working Well:
Process maturity will be validated over multiple sprints as routing–caching integration and concurrency testing introduce higher complexity.

4.8 Summary Table:

Alpha	State after Sprint 1
Opportunity	*Viable*
Stakeholders	*In Agreement*
Requirements	*Coherent*
Software System	*Architecture Selected*
Work	*Under Control*
Team	*Collaborating*
Way of Working	*In Place*